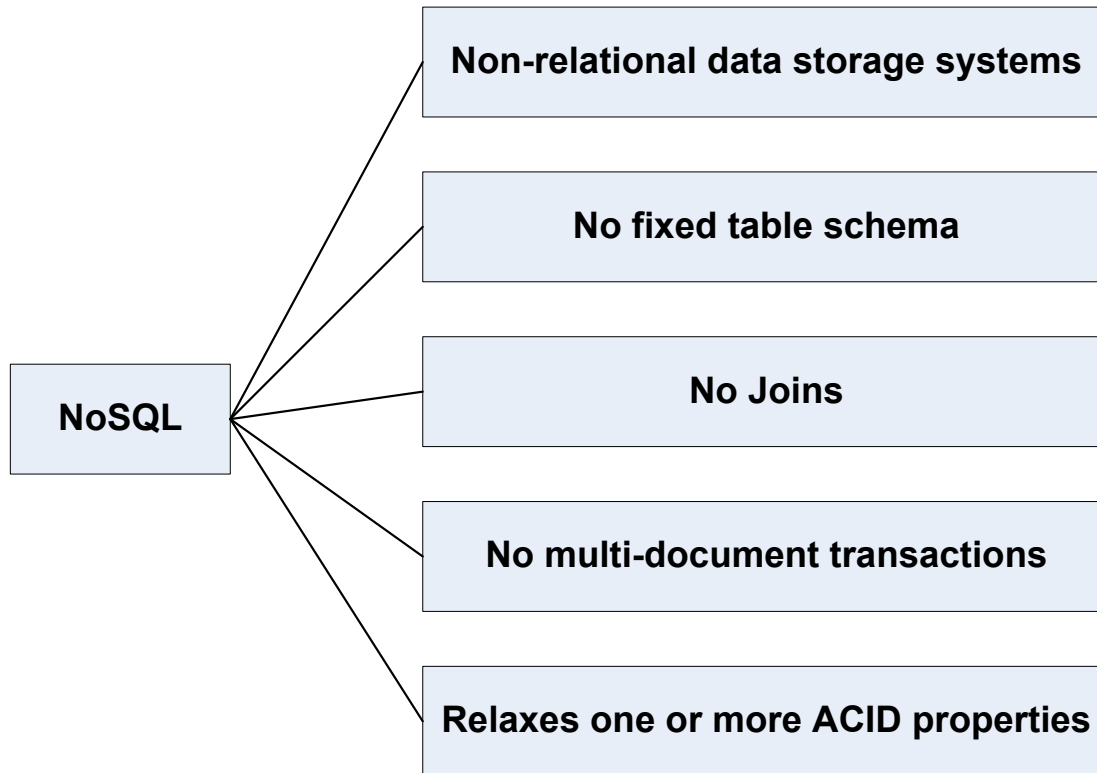




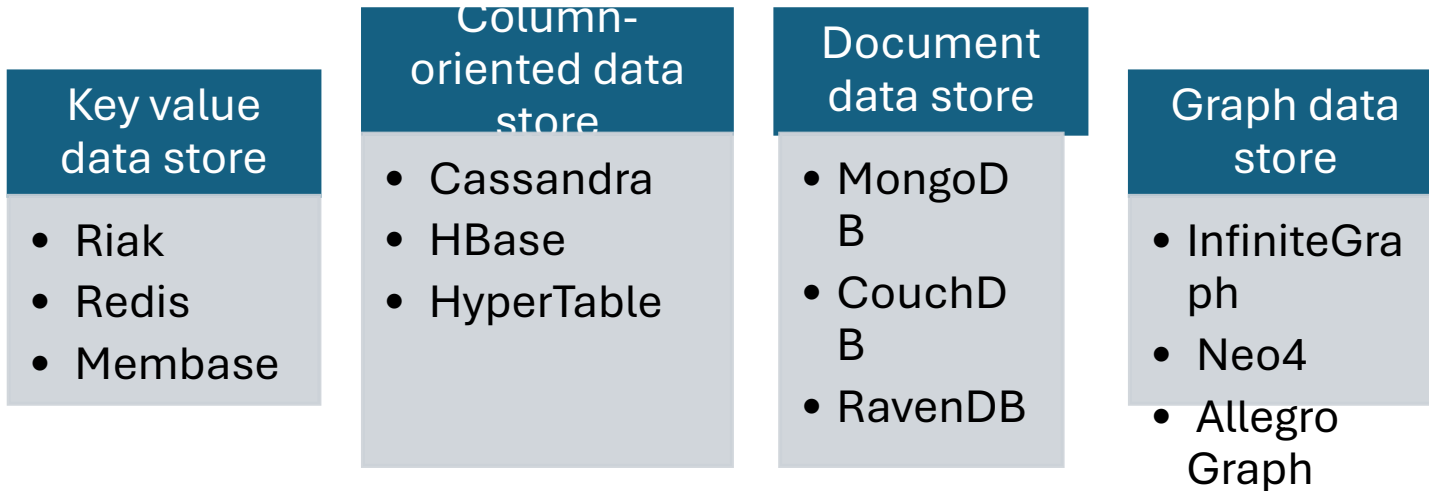
SDL- NoSQL & Query Processing

Module 1

What is NoSQL (Not Only SQL) ?

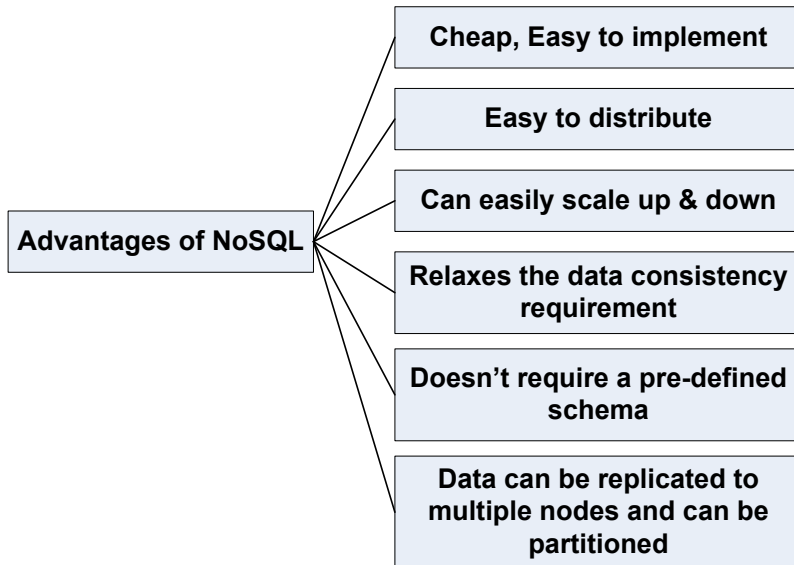


Types of NoSQL



- Key-Value
 - Shopping carts
 - Web user data analysis
 - Amazon, LinkedIn
- Document based
 - Real-time Analysis
 - Logging
 - Document archive management
- ▶ Column-oriented
 - ▶ Analyze huge web user actions
 - ▶ Sensor feeds
 - ▶ Facebook, Twitter, eBay, Netflix
- ▶ Graph-based
 - ▶ Network modeling
 - ▶ Recommendation
 - ▶ Walmart-upsell, cross-sell

Advantages & Disadvantages of NoSQL



Disadvantages of NoSQL

- Don't fully support relational features
 - no join, group by, order by operations (except within partitions)
 - no referential integrity constraints across partitions
- No declarative query language (e.g., SQL) → more programming
- Relaxed ACID (see CAP theorem) → fewer guarantees
- No easy integration with other applications that support SQL

SQL Vs. NoSQL

SQL	NoSQL
Relational database	Non-relational, distributed database
Relational model	Model-less approach
Pre-defined schema	Dynamic schema for unstructured data
Table based databases	Document-based or graph-based or wide column store or key-value pairs databases
Vertically scalable (by increasing system resources)	Horizontally scalable (by creating a cluster of commodity machines)
Uses SQL	Uses UnQL (Unstructured Query Language)
Not preferred for large datasets	Largely preferred for large datasets
Not a best fit for hierarchical data	Best fit for hierarchical storage as it follows the key-value pair of storing data similar to JSON (Java Script Object Notation)
Emphasis on ACID properties	Follows Brewer's CAP theorem
Excellent support from vendors	Relies heavily on community support
Supports complex querying and data keeping needs	Does not have good support for complex querying
Can be configured for strong consistency	Few support strong consistency (e.g., MongoDB), few others can be configured for eventual consistency (e.g., Cassandra)
Examples: Oracle, DB2, MySQL, MS SQL, PostgreSQL, etc.	MongoDB, HBase, Cassandra, Redis, Neo4j, CouchDB, Couchbase, Riak, etc.



Chapter 15: Query Processing

Database System Concepts, 7th Ed.

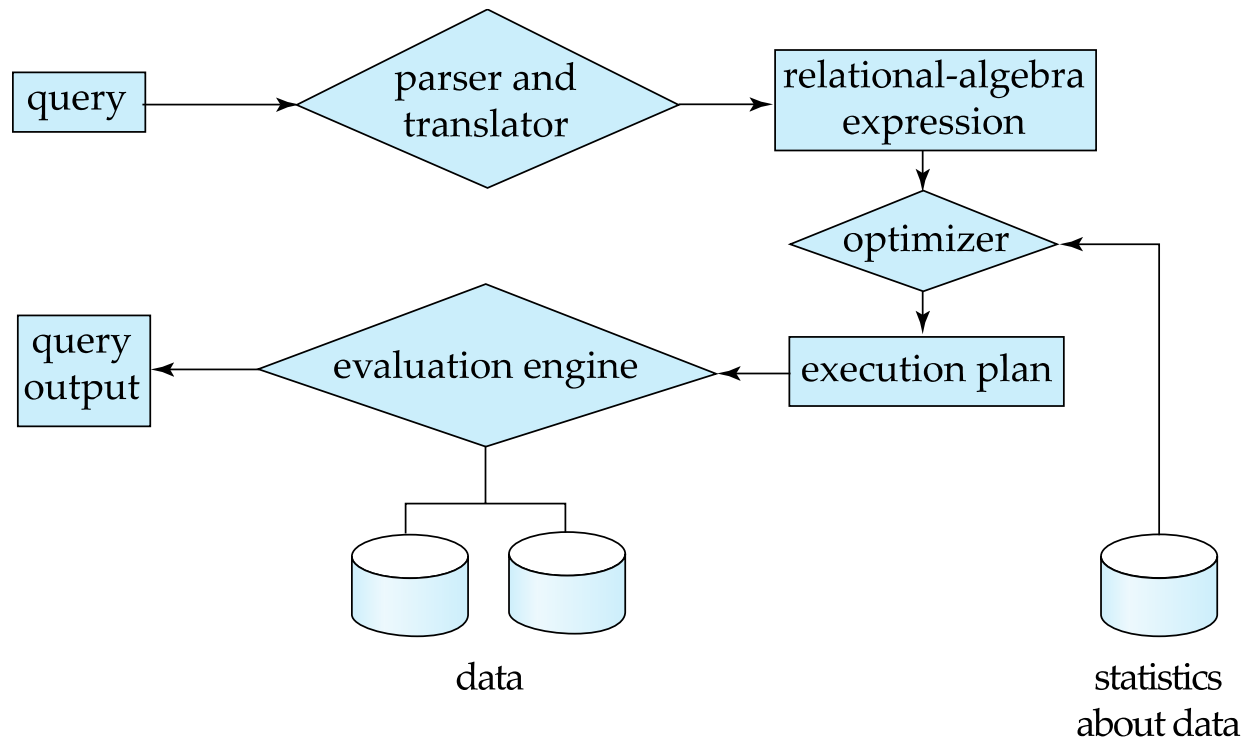
©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Basic Steps in Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation





Basic Steps in Query Processing (Cont.)

- Parsing and translation
 - translate the query into its internal form. This is then translated into relational algebra.
 - Parser checks syntax, verifies relations
- Evaluation
 - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.



Basic Steps in Query Processing: Optimization

- A relational algebra expression may have many equivalent expressions
 - E.g., $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$ is equivalent to $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$
- Each relational algebra operation can be evaluated using one of several different algorithms
 - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**. E.g.,:
 - Use an index on *salary* to find instructors with salary < 75000,
 - Or perform complete relation scan and discard instructors with salary ≥ 75000



Basic Steps: Optimization (Cont.)

- **Query Optimization:** Amongst all equivalent evaluation plans choose the one with lowest cost.
 - Cost is estimated using statistical information from the database catalog
 - e.g.. number of tuples in each relation, size of tuples, etc.
- In this chapter we study
 - How to measure query costs
 - Algorithms for evaluating relational algebra operations
 - How to combine algorithms for individual operations in order to evaluate a complete expression
- In Chapter 16
 - We study how to optimize queries, that is, how to find an evaluation plan with lowest estimated cost



Measures of Query Cost

- Many factors contribute to time cost
 - *disk access, CPU, and network communication*
- Cost can be measured based on
 - **response time**, i.e. total elapsed time for answering query, or
 - total **resource consumption**
- We use total resource consumption as cost metric
 - Response time harder to estimate, and minimizing resource consumption is a good idea in a shared database
- We ignore CPU costs for simplicity
 - Real systems do take CPU cost into account
 - Network costs must be considered for parallel systems
- We describe how estimate the cost of each operation
 - We do not include cost to writing output to disk



Measures of Query Cost

- Disk cost can be estimated as:
 - Number of seeks * average-seek-cost
 - Number of blocks read * average-block-read-cost
 - Number of blocks written * average-block-write-cost
- For simplicity we just use the **number of block transfers** *from disk and the number of seeks* as the cost measures
 - t_T – time to transfer one block
 - Assuming for simplicity that write cost is same as read cost
 - t_S – time for one seek
 - Cost for b block transfers plus S seeks
$$b * t_T + S * t_S$$
- t_S and t_T depend on where data is stored; with 4 KB blocks:
 - High end magnetic disk: $t_S = 4$ msec and $t_T = 0.1$ msec
 - SSD: $t_S = 20$ -90 microsec and $t_T = 2$ -10 microsec for 4KB



Measures of Query Cost (Cont.)

- Required data may be buffer resident already, avoiding disk I/O
 - But hard to take into account for cost estimation
- Several algorithms can reduce disk IO by using extra buffer space
 - Amount of real memory available to buffer depends on other concurrent queries and OS processes, known only during execution
- Worst case estimates assume that no data is initially in buffer and only the minimum amount of memory needed for the operation is available
 - But more optimistic estimates are used in practice



Selection Operation

- **File scan**
- Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.
 - Cost estimate = b_r block transfers + 1 seek
 - b_r denotes number of blocks containing records from relation r
 - If selection is on a key attribute, can stop on finding record
 - cost = $(b_r/2)$ block transfers + 1 seek
 - Linear search can be applied regardless of
 - selection condition or
 - ordering of records in the file, or
 - availability of indices
- Note: binary search generally does not make sense since data is not stored consecutively
 - except when there is an index available,
 - and binary search requires more seeks than index search



Selections Using Indices

- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A2 (clustering index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition
 - $Cost = (h_i + 1) * (t_T + t_S)$
- **A3 (clustering index, equality on nonkey)** Retrieve multiple records.
 - Records will be on consecutive blocks
 - Let b = number of blocks containing matching records
 - $Cost = h_i * (t_T + t_S) + t_S + t_T * b$



Selections Using Indices

- **A4 (secondary index, equality on key/non-key).**
 - Retrieve a single record if the search-key is a candidate key
 - $Cost = (h_i + 1) * (t_T + t_S)$
 - Retrieve multiple records if search-key is not a candidate key
 - each of n matching records may be on a different block
 - $Cost = (h_i + n) * (t_T + t_S)$
 - Can be very expensive!



Selections Involving Comparisons

- Can implement selections of the form $\sigma_{A \leq V}(r)$ or $\sigma_{A \geq V}(r)$ by using
 - a linear file scan,
 - or by using indices in the following ways:
- **A5 (clustering index, comparison).** (Relation is sorted on A)
 - For $\sigma_{A \geq V}(r)$ use index to find first tuple $\geq v$ and scan relation sequentially from there
 - For $\sigma_{A \leq V}(r)$ just scan relation sequentially till first tuple $> v$; do not use index
- **A6 (clustering index, comparison).**
 - For $\sigma_{A \geq V}(r)$ use index to find first index entry $\geq v$ and scan index sequentially from there, to find pointers to records.
 - For $\sigma_{A \leq V}(r)$ just scan leaf pages of index finding pointers to records, till first entry $> v$
 - In either case, retrieve records that are pointed to
 - requires an I/O per record; Linear file scan may be cheaper!

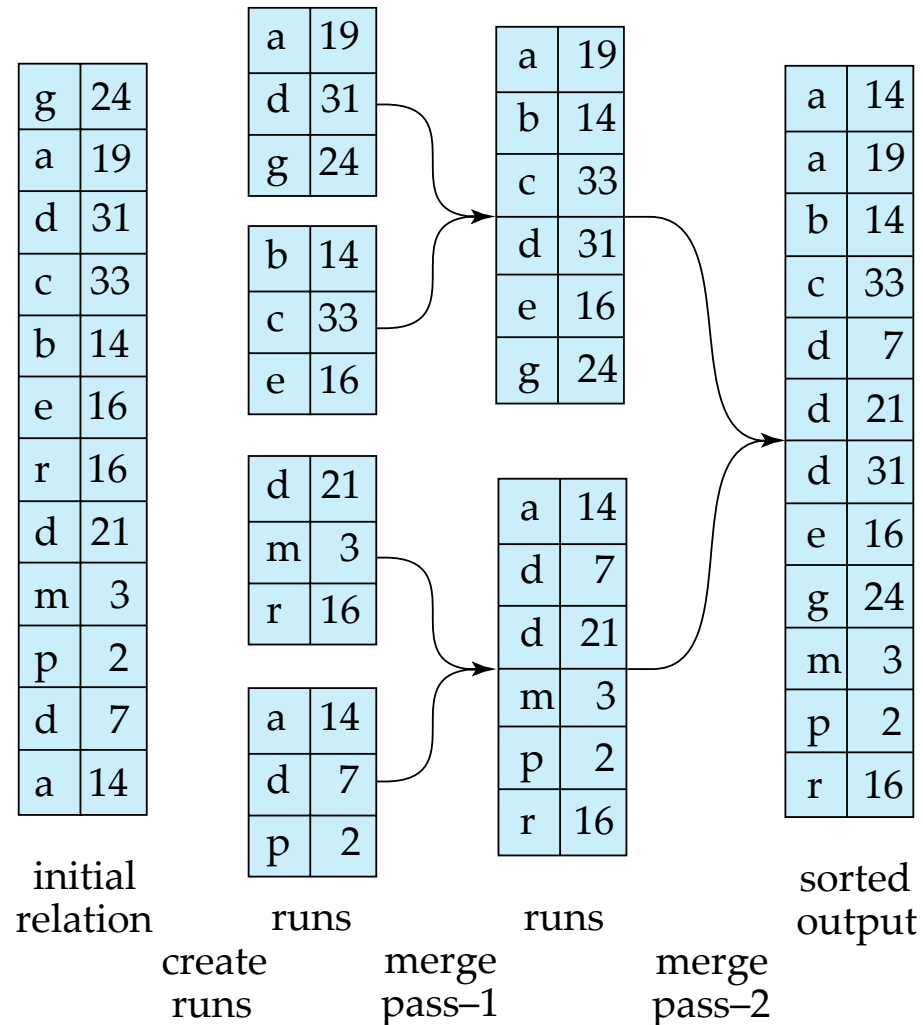


Sorting

- We may build an index on the relation, and then use the index to read the relation in sorted order. May lead to one disk block access for each tuple.
- For relations that fit in memory, techniques like quicksort can be used.
 - For relations that don't fit in memory, **external sort-merge** is a good choice.



Example: External Sorting Using Sort-Merge





External Sort-Merge

Let M denote memory size (in pages).

1. **Create sorted runs.** Let i be 0 initially.

Repeatedly do the following till the end of the relation:

- (a) Read M blocks of relation into memory
- (b) Sort the in-memory blocks
- (c) Write sorted data to run R_i ; increment i .

Let the final value of i be N

2. *Merge the runs (next slide).....*



External Sort-Merge (Cont.)

2. **Merge the runs (N-way merge).** We assume (for now) that $N < M$.
 1. Use N blocks of memory to buffer input runs, and 1 block to buffer output. Read the first block of each run into its buffer page
 2. **repeat**
 1. Select the first record (in sort order) among all buffer pages
 2. Write the record to the output buffer. If the output buffer is full write it to disk.
 3. Delete the record from its input buffer page.
If the buffer page becomes empty **then**
 read the next block (if any) of the run into the buffer.
 3. **until** all input buffer pages are empty:



External Sort-Merge (Cont.)

- If $N \geq M$, several merge *passes* are required.
 - In each pass, contiguous groups of $M - 1$ runs are merged.
 - A pass reduces the number of runs by a factor of $M - 1$, and creates runs longer by the same factor.
 - E.g. If $M=11$, and there are 90 runs, one pass reduces the number of runs to 9, each 10 times the size of the initial runs
 - Repeated passes are performed till all runs have been merged into one.



External Merge Sort (Cont.)

- Cost analysis:
 - 1 block per run leads to too many seeks during merge
 - Instead use b_b buffer blocks per run
 - ➔ read/write b_b blocks at a time
 - Can merge $\lfloor M/b_b \rfloor - 1$ runs in one pass
 - Total number of merge passes required: $\lceil \log_{\lfloor M/b_b \rfloor - 1} (b_r/M) \rceil$.
 - Block transfers for initial run creation as well as in each pass is $2b_r$
 - for final pass, we don't count write cost
 - we ignore final write cost for all operations since the output of an operation may be sent to the parent operation without being written to disk
 - Thus total number of block transfers for external sorting:
$$b_r (2 \lceil \log_{\lfloor M/b_b \rfloor - 1} (b_r / M) \rceil + 1)$$
 - Seeks: next slide



External Merge Sort (Cont.)

- Cost of seeks
 - During run generation: one seek to read each run and one seek to write each run
 - $2 \lceil b_r / M \rceil$
 - During the merge phase
 - Need $2 \lceil b_r / b_b \rceil$ seeks for each merge pass
 - except the final one which does not require a write
 - Total number of seeks:
$$2 \lceil b_r / M \rceil + \lceil b_r / b_b \rceil (2 \lceil \log_{\lfloor M/b_b \rfloor} (b_r / M) \rceil - 1)$$



Join Operation

- Several different algorithms to implement joins
 - Nested-loop join
 - Block nested-loop join
 - Indexed nested-loop join
 - Merge-join
 - Hash-join
- Choice based on cost estimate
- Examples use the following information
 - Number of records of *student*: 5,000 *takes*: 10,000
 - Number of blocks of *student*: 100 *takes*: 400



Nested-Loop Join

- To compute the theta join $r \bowtie_{\theta} s$
 for each tuple t_r **in** r **do begin**
 for each tuple t_s **in** s **do begin**
 test pair (t_r, t_s) to see if they satisfy the join condition θ
 if they do, add $t_r \cdot t_s$ to the result.
 end
 end
- r is called the **outer relation** and s the **inner relation** of the join.
- Requires no indices and can be used with any kind of join condition.
- Expensive since it examines every pair of tuples in the two relations.



Chapter 15: Query Processing

Database System Concepts, 7th Ed.

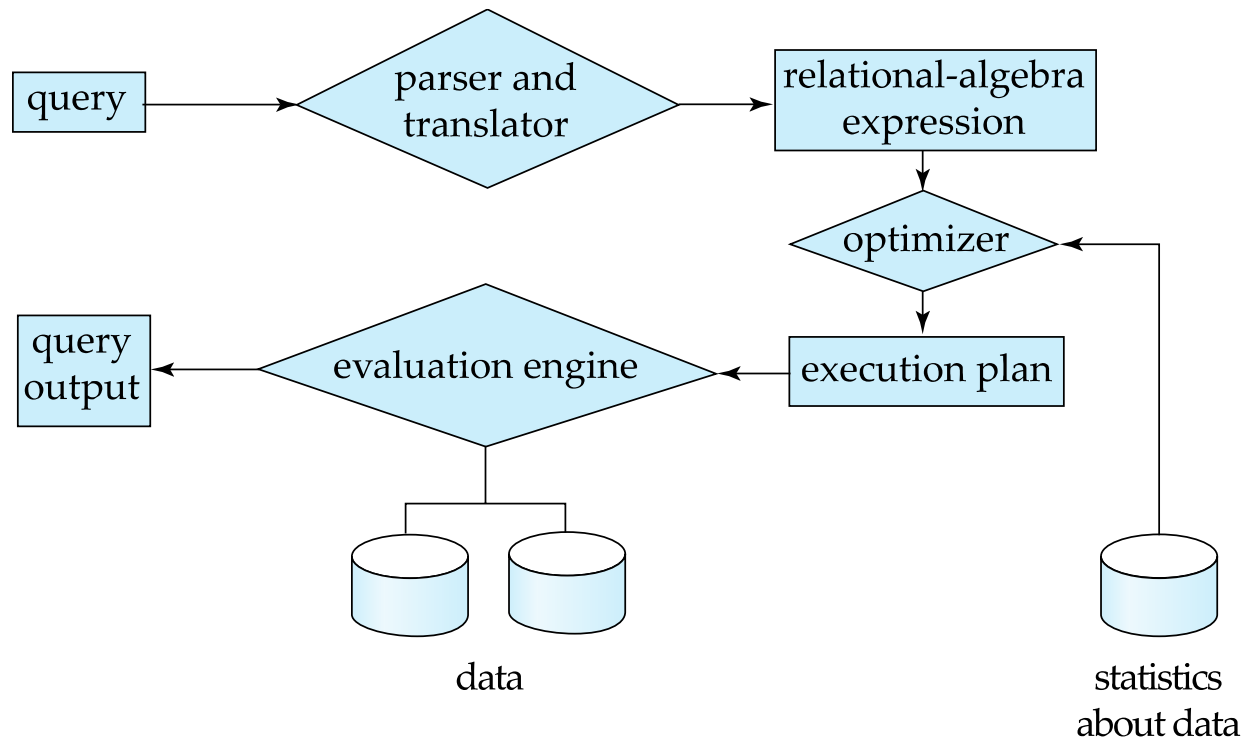
©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Basic Steps in Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation





Basic Steps in Query Processing (Cont.)

- Parsing and translation
 - translate the query into its internal form. This is then translated into relational algebra.
 - Parser checks syntax, verifies relations
- Evaluation
 - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.



Basic Steps in Query Processing: Optimization

- A relational algebra expression may have many equivalent expressions
 - E.g., $\sigma_{\text{salary} < 75000}(\Pi_{\text{salary}}(\text{instructor}))$ is equivalent to $\Pi_{\text{salary}}(\sigma_{\text{salary} < 75000}(\text{instructor}))$
- Each relational algebra operation can be evaluated using one of several different algorithms
 - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**. E.g.,:
 - Use an index on *salary* to find instructors with salary < 75000,
 - Or perform complete relation scan and discard instructors with salary ≥ 75000



Basic Steps: Optimization (Cont.)

- **Query Optimization:** Amongst all equivalent evaluation plans choose the one with lowest cost.
 - Cost is estimated using statistical information from the database catalog
 - e.g.. number of tuples in each relation, size of tuples, etc.
- In this chapter we study
 - How to measure query costs
 - Algorithms for evaluating relational algebra operations
 - How to combine algorithms for individual operations in order to evaluate a complete expression
- In Chapter 16
 - We study how to optimize queries, that is, how to find an evaluation plan with lowest estimated cost



Measures of Query Cost

- Many factors contribute to time cost
 - *disk access, CPU, and network communication*
- Cost can be measured based on
 - **response time**, i.e. total elapsed time for answering query, or
 - total **resource consumption**
- We use total resource consumption as cost metric
 - Response time harder to estimate, and minimizing resource consumption is a good idea in a shared database
- We ignore CPU costs for simplicity
 - Real systems do take CPU cost into account
 - Network costs must be considered for parallel systems
- We describe how estimate the cost of each operation
 - We do not include cost to writing output to disk



Measures of Query Cost

- Disk cost can be estimated as:
 - Number of seeks * average-seek-cost
 - Number of blocks read * average-block-read-cost
 - Number of blocks written * average-block-write-cost
- For simplicity we just use the **number of block transfers** *from disk and the number of seeks* as the cost measures
 - t_T – time to transfer one block
 - Assuming for simplicity that write cost is same as read cost
 - t_S – time for one seek
 - Cost for b block transfers plus S seeks
$$b * t_T + S * t_S$$
- t_S and t_T depend on where data is stored; with 4 KB blocks:
 - High end magnetic disk: $t_S = 4$ msec and $t_T = 0.1$ msec
 - SSD: $t_S = 20$ -90 microsec and $t_T = 2$ -10 microsec for 4KB



Measures of Query Cost (Cont.)

- Required data may be buffer resident already, avoiding disk I/O
 - But hard to take into account for cost estimation
- Several algorithms can reduce disk IO by using extra buffer space
 - Amount of real memory available to buffer depends on other concurrent queries and OS processes, known only during execution
- Worst case estimates assume that no data is initially in buffer and only the minimum amount of memory needed for the operation is available
 - But more optimistic estimates are used in practice



Selection Operation

- **File scan**
- Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.
 - Cost estimate = b_r block transfers + 1 seek
 - b_r denotes number of blocks containing records from relation r
 - If selection is on a key attribute, can stop on finding record
 - cost = $(b_r/2)$ block transfers + 1 seek
 - Linear search can be applied regardless of
 - selection condition or
 - ordering of records in the file, or
 - availability of indices
- Note: binary search generally does not make sense since data is not stored consecutively
 - except when there is an index available,
 - and binary search requires more seeks than index search



Selections Using Indices

- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A2 (clustering index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition
 - $Cost = (h_i + 1) * (t_T + t_S)$
- **A3 (clustering index, equality on nonkey)** Retrieve multiple records.
 - Records will be on consecutive blocks
 - Let b = number of blocks containing matching records
 - $Cost = h_i * (t_T + t_S) + t_S + t_T * b$



Selections Using Indices

- **A4 (secondary index, equality on key/non-key).**
 - Retrieve a single record if the search-key is a candidate key
 - $Cost = (h_i + 1) * (t_T + t_S)$
 - Retrieve multiple records if search-key is not a candidate key
 - each of n matching records may be on a different block
 - $Cost = (h_i + n) * (t_T + t_S)$
 - Can be very expensive!



Selections Involving Comparisons

- Can implement selections of the form $\sigma_{A \leq V}(r)$ or $\sigma_{A \geq V}(r)$ by using
 - a linear file scan,
 - or by using indices in the following ways:
- **A5 (clustering index, comparison).** (Relation is sorted on A)
 - For $\sigma_{A \geq V}(r)$ use index to find first tuple $\geq v$ and scan relation sequentially from there
 - For $\sigma_{A \leq V}(r)$ just scan relation sequentially till first tuple $> v$; do not use index
- **A6 (clustering index, comparison).**
 - For $\sigma_{A \geq V}(r)$ use index to find first index entry $\geq v$ and scan index sequentially from there, to find pointers to records.
 - For $\sigma_{A \leq V}(r)$ just scan leaf pages of index finding pointers to records, till first entry $> v$
 - In either case, retrieve records that are pointed to
 - requires an I/O per record; Linear file scan may be cheaper!

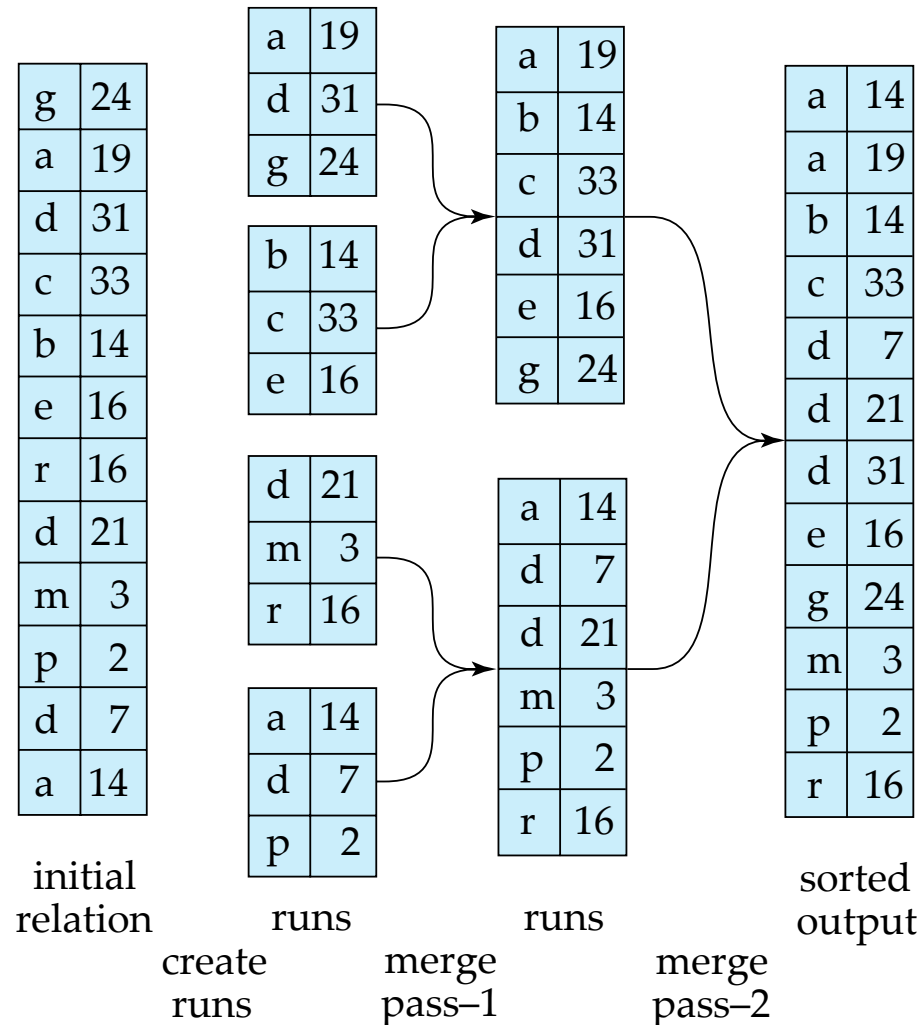


Sorting

- We may build an index on the relation, and then use the index to read the relation in sorted order. May lead to one disk block access for each tuple.
- For relations that fit in memory, techniques like quicksort can be used.
 - For relations that don't fit in memory, **external sort-merge** is a good choice.



Example: External Sorting Using Sort-Merge





External Sort-Merge

Let M denote memory size (in pages).

1. **Create sorted runs.** Let i be 0 initially.

Repeatedly do the following till the end of the relation:

- (a) Read M blocks of relation into memory
- (b) Sort the in-memory blocks
- (c) Write sorted data to run R_i ; increment i .

Let the final value of i be N

2. *Merge the runs (next slide).....*



External Sort-Merge (Cont.)

2. **Merge the runs (N-way merge).** We assume (for now) that $N < M$.
 1. Use N blocks of memory to buffer input runs, and 1 block to buffer output. Read the first block of each run into its buffer page
 2. **repeat**
 1. Select the first record (in sort order) among all buffer pages
 2. Write the record to the output buffer. If the output buffer is full write it to disk.
 3. Delete the record from its input buffer page.
If the buffer page becomes empty **then**
 read the next block (if any) of the run into the buffer.
 3. **until** all input buffer pages are empty:



External Sort-Merge (Cont.)

- If $N \geq M$, several merge *passes* are required.
 - In each pass, contiguous groups of $M - 1$ runs are merged.
 - A pass reduces the number of runs by a factor of $M - 1$, and creates runs longer by the same factor.
 - E.g. If $M=11$, and there are 90 runs, one pass reduces the number of runs to 9, each 10 times the size of the initial runs
 - Repeated passes are performed till all runs have been merged into one.



External Merge Sort (Cont.)

- Cost analysis:
 - 1 block per run leads to too many seeks during merge
 - Instead use b_b buffer blocks per run
 - ➔ read/write b_b blocks at a time
 - Can merge $\lfloor M/b_b \rfloor - 1$ runs in one pass
 - Total number of merge passes required: $\lceil \log_{\lfloor M/b_b \rfloor - 1} (b_r/M) \rceil$.
 - Block transfers for initial run creation as well as in each pass is $2b_r$
 - for final pass, we don't count write cost
 - we ignore final write cost for all operations since the output of an operation may be sent to the parent operation without being written to disk
 - Thus total number of block transfers for external sorting:
$$b_r (2 \lceil \log_{\lfloor M/b_b \rfloor - 1} (b_r / M) \rceil + 1)$$
 - Seeks: next slide



External Merge Sort (Cont.)

- Cost of seeks
 - During run generation: one seek to read each run and one seek to write each run
 - $2 \lceil b_r / M \rceil$
 - During the merge phase
 - Need $2 \lceil b_r / b_b \rceil$ seeks for each merge pass
 - except the final one which does not require a write
 - Total number of seeks:
$$2 \lceil b_r / M \rceil + \lceil b_r / b_b \rceil (2 \lceil \log_{\lfloor M/b_b \rfloor} (b_r / M) \rceil - 1)$$



Join Operation

- Several different algorithms to implement joins
 - Nested-loop join
 - Block nested-loop join
 - Indexed nested-loop join
 - Merge-join
 - Hash-join
- Choice based on cost estimate
- Examples use the following information
 - Number of records of *student*: 5,000 *takes*: 10,000
 - Number of blocks of *student*: 100 *takes*: 400



Nested-Loop Join

- To compute the theta join $r \bowtie_{\theta} s$
 for each tuple t_r **in** r **do begin**
 for each tuple t_s **in** s **do begin**
 test pair (t_r, t_s) to see if they satisfy the join condition θ
 if they do, add $t_r \cdot t_s$ to the result.
 end
 end
- r is called the **outer relation** and s the **inner relation** of the join.
- Requires no indices and can be used with any kind of join condition.
- Expensive since it examines every pair of tuples in the two relations.